面向多任务的机械臂智能运动规划方法研究

学生姓名：陈雄杰
学号：2022301606
专业：飞行器制造工程
班级：05012201
指导教师：宗立军

摘要

随着制造业由大批量、单品种、长周期生产方式向多品种、小批量、快切换的柔性制造模式转变，工业机械臂所承担的任务类型与作业环境都发生了显著变化。
传统工业机器人控制系统通常围绕某一固定工艺过程构建，其底层逆运动学求解、上层动作编排以及末端执行接口往往采用紧耦合设计，因此在面对任务切换、目标变化和工位重构时容易出现复用性不足、扩展代价高和系统维护困难等问题。
针对上述问题，本文围绕“面向多任务的机械臂智能运动规划方法研究”这一课题展开系统设计与实现，以 UR10e 六自由度机械臂为研究对象，构建了一套融合图神经网络逆运动学求解、视觉目标定位、子任务原语封装、行为树调度以及可插拔执行桥接机制的多任务运动规划系统。

本文首先对机械臂多任务运动规划问题进行形式化描述，将复杂任务的执行过程划分为感知层、求解层、原语层、调度层和执行层五个部分。
在求解层中，针对传统解析法和数值法对机械臂构型适应性不足、在复杂多任务场景下难以快速复用的问题，本文将 UR10e 关节链抽象为

图结构，以关节为节点、相邻连杆连接关系为边，构建适用于图神经网络建模的状态表示方式。
在节点特征设计方面，综合引入关节角的正余弦周期编码、各关节在当前姿态下的位置与姿态、目标末端位姿与各关节局部位姿之间的相对误差等信息，从而增强模型对链式拓扑和空间几何关系的表达能力。
在网络结构方面，本文基于 GATv2 构建逆运动学预测模型，并通过多层感知机解码头输出各关节的角度增量。

为提升模型结果的物理一致性与训练稳定性，本文在训练阶段引入可微正运动学约束，利用基于 D-H 参数构建的可微正运动学模块，将网络预测结果重新映射到末端位姿空间。
在损失函数设计中，同时考虑关节空间误差、末端位置误差和末端姿态误差三类指标，并采用基于不确定性的自动加权多任务损失机制实现各分量权重的自适应平衡。
在推理阶段，本文将训练后的网络封装为迭代式求解器，在每次迭代中结合步长裁剪、关节限位约束、姿态收敛判定和 MuJoCo 前向验证实现稳定求解，从而保证网络输出不仅具备数据驱动模型的非线性拟合能力，同时兼顾工业控制场景对物理合理性和执行安全性的要求。

在系统工程实现方面，本文基于 MuJoCo 构建了 UR10e 机械臂的仿真环境，完成了训练数据自动生成、数据集标准化、模型训练与保存、求解器封装、视觉目标估计和子任务库构建等工作。
系统形成了 `Approach`、`Orient`、`Reach`、`Linear`、`Grasp`、`Release`、`Retreat`、`Hover`、`Arc` 和 `Screw` 共 10 类子任务原语，并基于行为树实现了抓取转运、视觉巡检和装配旋拧三类复合任务的流程重组与统一调度。
为保证后续能够平滑接入真实机械臂，本文将底层执行部分进一步抽象为运行时桥接层，由 `runtime_bridge.py` 统一负责关节状态写入、MuJoCo 前向更新、末端位姿读取和命令发射，由 `robot_ip_client.py` 对接机械臂指令接口。

考虑到当前尚处于仿真与预联调阶段，本文没有直接向真实机械臂发送网络指令，而是先将 `movej(...)` 格式的 URScript 风格命令逐行写入文本文件，用于检验动作生成逻辑和通信层接口设计的正确性，为后续以 IP 通信方式接入真实设备预留明确的替换位置。

在实验验证方面，本文围绕任务书和开题报告中提出的核心要求，重点考察系统的多任务重构能力、子任务复用能力、任务链执行稳定性以及执行接口的工程可扩展性。
在当前代码版本下，三类行为树任务均已在 MuJoCo 环境中完成联调验证。
其中，抓取转运任务连续三次运行均成功完成，放置点末端误差为 0.0 mm，单次任务输出 82 行 `movej` 命令字符串；视觉巡检任务连续三次运行均成功完成，回到 home 位置的末端误差为 2.5638 mm，单次任务输出 72 行命令；装配旋拧任务连续三次运行均成功完成，任务末端回到 safe-home 的误差为 3.0453 mm，单次任务输出 164 行命令，旋拧阶段最终位置误差为 1.72 mm，旋转矩阵均方误差为 0.00002。
上述结果表明，本文构建的系统能够在统一逆运动学求解器和统一子任务库支持下完成三类典型复合任务，具备较好的工程闭环能力和后续实机扩展基础。

综上，本文工作并非仅停留在单一算法精度验证层面，而是围绕“多任务规划、子任务复用、统一求解和可插拔执行接口”这一主线，构建了从模型到系统、从仿真到实机过渡层的完整技术路线。
相关研究成果可为柔性制造、视觉引导操作、智能装配和模块化机械臂任务重构提供一定的理论与工程参考。

关键词：机械臂；多任务运动规划；图神经网络；逆运动学；行为树；视觉定位；MuJoCo；IP 通信

Abstract

With the development of flexible manufacturing, industrial robotic manipulators are required to handle diversified tasks rather than fixed single-process operations.

Conventional robot systems are usually designed around tightly coupled control logic, which makes it difficult to reuse the low-level solver, reorganize action sequences, and extend the execution interface when the task or environment changes.

To address these issues, this thesis studies an intelligent motion-planning framework for multi-task robotic manipulation.

The UR10e six-degree-of-freedom manipulator is selected as the research platform, and a complete system integrating graph-neural-network-based inverse kinematics, vision-based target localization, reusable motion primitives, behavior-tree scheduling, and a pluggable runtime bridge is constructed.

First, the manipulator kinematic chain is represented as a graph, where joints are treated as nodes and physical connections are treated as edges.

Node features are composed of sinusoidal joint-angle encoding, current joint positions and orientations, and relative errors between local joint states and the target end-effector pose.

Based on this representation, a GATv2-based inverse-kinematics model is designed to predict joint-angle increments.

To enforce physical consistency, a differentiable forward-kinematics module is introduced during model training.

An uncertainty-based automatic weighting strategy is adopted so that the model can dynamically balance multiple loss components.

Second, the trained network is encapsulated into an iterative solver with step clipping, joint-range projection, forward validation, and convergence checking, which improves robustness for online calls in simulation tasks.

Third, at the system level, a MuJoCo-based UR10e simulation environment is established.
Ten reusable subtasks are implemented, including `Approach`, `Orient`, `Reach`, `Linear`, `Grasp`, `Release`, `Retreat`, `Hover`, `Arc`, and `Screw`.
Three representative compound tasks are then organized by behavior trees: grasp-and-transfer, visual inspection, and screw-in assembly.
At the current stage, the generated URScript-style `movej(...)` commands are written line by line to text files, which makes it convenient to verify the action stream before replacing the file-writing logic with IP-based command transmission.

Finally, task-level simulation experiments are carried out based on the current codebase.
The three behavior-tree tasks are all successfully executed in MuJoCo.
The grasp-and-transfer task succeeds in three repeated runs with a final placement error of 0.0 mm and 82 command lines per run.
The visual-inspection task succeeds in three repeated runs with a final home-position error of 2.5638 mm and 72 command lines per run.
The screw-in assembly task also succeeds in three repeated runs, with a final safe-home error of 3.0453 mm, 164 command lines per run, a screw-stage position error of 1.72 mm, and a rotation-matrix mean square error of 0.00002.
These results demonstrate that the proposed framework can support multi-task reconstruction with a unified solver, a shared primitive library, and a replaceable execution interface.

Key words: robotic manipulator; multi-task motion planning; graph neural network; inverse kinematics; behavior tree; vision localization; MuJoCo; IP communication

---

第1章 绪论

1.1 研究背景

工业机器人是现代制造系统中最重要的自动化装备之一。
在早期的标准化生产场景中，机械臂通常承担点焊、搬运、喷涂和上下料等固定流程任务。
这类任务具有节拍稳定、工位固定、环境变化小和工艺重复度高等特点。
因此，传统控制系统大多围绕单一场景构建，底层运动求解器、上层流程逻辑和执行接口可以针对具体工艺进行深度定制。

随着制造业向柔性化和智能化方向发展，单一场景下的深度定制方案逐渐暴露出局限。
一方面，生产对象发生变化后，原有动作流程往往无法直接复用。
另一方面，当机械臂需要在抓取转运、视觉巡检、对准装配和旋拧执行等多种任务之间切换时，系统不仅要重新规划动作，还要协调感知结果、位姿求解、执行顺序和异常回退逻辑。
若系统设计仍保持强耦合，则每增加一种新任务，都可能带来底层逻辑复制、接口重写和联调成本上升等问题。

在这一背景下，“多任务机械臂控制”已不再是简单的任务堆叠问题，而是一个涉及任务分解、动作复用、统一求解和执行抽象的系统工程问题。
本科毕业设计的价值，也因此不应仅局限于验证某一单独算法的理论有效性，而应体现为对完整工程链路的理解、设计和实现能力。

本课题来源于真实工程需求。
根据任务书与开题报告，本研究需要面向复杂任务场景构建机械臂运动规划与控制系统，形成不少于 5 类可复用的子任务动作，实现不少于 3 类任务分解与重构流程，并围绕机械臂逆运动学求解、任务流程组织和系统级联调展开研究。

与此同时，开题报告还强调了“构型自适应”“可微正运动学约束”“图结构化建模”和“从算法到系统验证”的研究路线。
这意味着本文必须在方法设计与工程实现两个层面同时发力。

逆运动学求解是机械臂系统的底层核心环节。
对于给定的末端位置和姿态目标，系统必须快速求出可执行的关节角配置。
传统解析法在特定构型下求解速度快，但通用性有限；数值法适用范围较广，但在奇异位形附近容易出现收敛困难或对初值敏感。
当系统面对多任务切换时，仅有一个“能算出结果”的求解器并不足够，更重要的是求解器是否便于被不同任务调用、是否能在统一接口下运行、是否具备与视觉模块和调度模块协同工作的能力。

近年来，深度学习和图神经网络为机械臂运动学建模提供了新的研究方向。
与普通多层感知机相比，图神经网络能够更自然地表示机械臂的关节拓扑关系。
机械臂本身具有典型的链式结构，各关节之间的影响具有明确的相邻传播特性。
因此，将关节链抽象为图结构，再利用图注意力机制对节点之间的信息依赖进行建模，具有明确的结构合理性。
然而，若模型设计只追求离线预测误差，而忽略与正运动学约束、仿真环境和任务系统的结合，则其工程应用价值仍然有限。

同时，在多任务场景下，高层流程组织同样至关重要。
如果将每一种任务流程都写成一段独立脚本，虽然可以快速验证，但缺乏可维护性和复用能力。
相比之下，行为树通过条件节点、动作节点、顺序节点和回退节点描述任务逻辑，能够较好表达“条件判断—动作执行—异常处理—流程重组”的层次关系，非常适合用于多任务机械臂系统的高层调度。

因此，本文选择以图神经网络逆运动学求解为底层核心，以行为树为任务组织框架，以子任务原语为中间接口，并通过运行时桥接层隔离底层执行细节。
这一思路既回应了课题任务要求，也与开题报告中提出的研究主线保持一致。

1.2 研究意义

本文的研究意义可以从理论意义、工程意义和应用意义三个方面进行分析。

在理论意义方面，本文将机械臂逆运动学问题与图神经网络建模方法相结合，探索了从“机械结构拓扑”到“神经网络表示”的映射方式。
与将关节角简单视为向量输入的做法相比，本文强调以结构化方式组织节点和边，并通过目标位姿相对误差信息增强网络对任务目标的感知能力。
同时，本文引入可微正运动学约束，使神经网络输出能够回到物理空间接受末端位姿误差检验，从而实现数据驱动方法与物理先验之间的结合。
这种“灰箱化”建模思路，对于提升数据驱动机械臂求解模型的可解释性和工程可信度具有一定参考价值。

在工程意义方面，本文并未将研究停留在模型训练与离线测试层面，而是将数据生成、求解器封装、视觉定位、任务原语、行为树调度和执行桥接统一纳入同一项目框架。
这使得研究成果可以直接服务于系统联调，也更加符合本科毕业设计对于“能够独立完成一套系统方案”的要求。
尤其是运行时桥接层的加入，使底层执行接口与上层任务逻辑相分离，当前可通过文本文件输出机械臂命令，后续则可进一步替换为 IP 通信或实机驱动接口，具有较强的工程扩展潜力。

在应用意义方面，本文面向的抓取转运、视觉巡检和装配旋拧任务，具有典型的工业应用背景。
抓取转运是智能物流和柔性上下料中的基本环节；视觉巡检适用于质量检测与表面观察；旋拧装配则对应精密装配中的典型操作。
如果一套系统能够在统一求解器和统一子任务库支持下完成这些任务，说明其对多任务流程切换和任务重构具有一定适应能力。
这一能力对于未来的柔性制造单元和智能工作站具有现实意义。

1.3 国内外研究现状

传统逆运动学求解主要包括解析法和数值迭代法。
解析法在特定构型下求解速度快、结果明确，但对构型依赖强、泛化性差。
数值法适用范围较广，但通常依赖初值，在奇异位形附近容易出现收敛困难。
当研究目标从“单次求解是否准确”转向“系统是否便于多任务调用与重构”时，传统方法的局限性更加明显。

随着深度学习的发展，研究者开始尝试利用神经网络直接学习末端位姿到关节空间的映射关系。
但若忽视机械臂本身的链式结构和逆运动学多解性，纯数据驱动方法容易输出物理上不可执行的解。
图神经网络由于能够显式表达节点与边之间的关系，在机械臂关节链建模中表现出良好的适配性。
不过，已有研究中仍有不少工作主要关注模型离线精度，对系统级落地、任务组织和执行接口抽象涉及较少。

在高层任务组织方面，状态机、规划器与行为树均有应用。
状态机适合流程简单、分支较少的任务，但在多任务重构场景下容易出现状态膨胀。
行为树则强调节点职责清晰、行为组合灵活和失败回退可控，更适合用于本文关注的多任务机械臂系统。

综合来看，当前研究仍存在以下不足：底层逆运动学求解与高层任务调度之间缺乏统一接口；纯数据驱动方法往往忽略物理先验；针对多任务场景的研究偏少；感知、规划、执行之间往往分离实现，系统级闭环验证不足。

1.4 研究目标、主要内容与创新点

结合任务书要求、开题报告研究路线以及当前项目基础，本文确定如下研究目标：构建面向多任务场景的机械臂运动规划系统，实现从目标感知到动作输出的完整链路；设计基于图神经网络的逆运动学求解方法，提升底层求解器的结构表达能力和在线调用能力；建立可复用的子任务动作原语库，以支持多种复杂任务的流程拆解和重组；采用行为树完成高层任务编排，实现抓取转运、视觉巡检和装配旋拧三类任务的统一组织；构建可插拔执行桥接层，保证当前仿真阶段与后续实机 IP 通信阶段之间的平滑过渡。

围绕上述目标，本文主要工作包括：完成 UR10e 机械臂运动学建模与问题定义；构建链式图拓扑表示和图神经网络逆运动学模型；引入可微正运动学约束和自动加权多任务损失；将模型封装为可迭代调用的在线求解器；构建训练数据生成流程和模型训练流程；设计视觉定位模块和任务黑板数据组织方式；实现 10 类子任务原语；构建 3 类行为树复合任务；设计运行时桥接层和命令输出接口；在 MuJoCo 环境下对系统进行联调与任务级验证。

本文的创新点和工作特色主要体现在以下几个方面：将 UR10e 机械臂关节链抽象为图结构，并结合局部位姿与目标相对误差构建节点特征，使逆运动学求解过程能够显式利用机械臂的结构拓扑信息；在训练过程中引入可微正运动学模块，同时约束关节空间误差、末端位置误差和末端姿态误差，提升模型输出的物理一致性；采用基于不确定性的自动加权多任务损失机制，减轻不同量纲误差联合优化时对人工权重调节的依赖；将神经网络模型进一步封装为迭代式求解器，在推理阶段引入步长裁剪、关节限位和前向验证，增强在线调用的稳定

性和安全性；构建覆盖抓取、移动、对准、退让、悬停、圆弧扫描和旋拧操作的统一原语库，并基于行为树实现多任务流程重构；在工程实现上增加运行时桥接层，将关节状态应用、仿真前向计算和命令发射过程统一抽象，使机械臂控制逻辑与底层通信细节相互解耦。

1.5 论文结构安排

全文共分为六章，并附有参考文献与附录。

第1章为绪论，阐述研究背景、研究意义、国内外研究现状、研究目标、创新点和论文结构。

第2章为课题需求分析与总体方案设计，重点分析任务书指标、系统层次结构、关键模块关系和总体技术路线。

第3章为基于图神经网络的逆运动学建模与求解方法设计，重点给出机械臂图表示、节点特征构建、GATv2 求解网络、可微正运动学约束、自动加权多任务损失和迭代求解器设计。

第4章为系统工程实现与控制策略部署，详细介绍数据集生成、模型训练流程、求解器封装、视觉模块、子任务原语库、行为树组织方式以及运行时桥接层设计。

第5章为实验设计与结果分析，给出实验平台、评价指标、任务级验证结果、图表配置建议以及系统局限性分析。

第6章为总结与展望，对全文工作进行归纳，并讨论后续可在实机部署、通信接口、任务泛化和模型鲁棒性方面继续推进的方向。

1.6 本章小结

本章围绕课题来源、研究背景和国内外研究现状进行了阐述。

通过分析可以看出，面向多任务的机械臂系统需要同时解决底层求解、中层原语复用和高层任务重构三个层面的问题。

单纯强调某一算法精度不足以满足本科毕设对于完整工程系统的要求。

因此，本文将后续研究重点放在“统一求解器 + 统一原语库 + 统一任务组织 + 统一执行桥接”这一完整链路上。

【图1-1 课题研究背景示意图】
配图说明：本图建议采用“传统固定工艺机器人”与“面向多任务的柔性机器人系统”的对比示意方式绘制。左侧展示传统系统只能完成单一搬运或焊接流程，模块之间采用硬编码连接；右侧展示本文系统由视觉、GNN 求解器、原语库、行为树和执行桥接组成，并能切换抓取转运、视觉巡检和旋拧装配三类任务。图中应突出“任务切换”“模块复用”“接口解耦”三个关键词。

【图1-2 本文研究内容与章节关系图】
配图说明：可采用流程框图形式绘制。从第1章问题提出开始，逐步连接到第2章总体方案、第3章求解方法、第4章工程实现、第5章实验验证、第6章总结展望。每个章节框内写明核心内容，有利于开篇帮助读者建立整体认知。

第2章 课题需求分析与总体方案设计

2.1 任务书与开题报告要求分析

课题需求分析是系统设计的前提。
若不先明确本科毕业设计的目标边界，则后续方法设计和实验安排容易偏离主题。

根据任务书，本课题聚焦于机械臂复杂任务分解、子任务运动规划与控制、多任务离线重构控制等内容。
这说明课题核心并不是单纯训练一个逆运动学模型，也不是孤立地实现某一个演示脚本，而是要围绕“任务如何拆解、动作如何复用、流程如何重组”建立完整方案。

任务书中对系统能力提出了较为明确的指标要求。
第一，需要具备不少于 5 类可复用子任务。

第二，需要实现不少于 3 类任务分解与重构场景。
第三，在子任务控制精度方面应达到较高水平，尤其是关键执行环节应接近毫米级控制精度。
第四，系统应形成从输入到输出的完整链路，而不仅是停留在离线仿真或单点数据测试阶段。

开题报告进一步提出了研究思路。
其一，强调“构型自适应能力”，说明系统设计不宜过度依赖某一固定场景下的人工解析过程。
其二，强调“可微正运动学计算作为强制性约束嵌入网络”，说明模型设计必须体现物理先验的引入。
其三，强调“图结构化表征与几何感知型 GNN”，说明底层逆运动学建模应该突出结构表达而非简单黑箱映射。
其四，强调“从算法性能、任务完成到系统集成的多层次评估”，说明实验设计不能只给出模型损失下降曲线，还要展示任务级联调效果。

由此可见，本课题需要同时满足算法层、系统层和验证层三类要求。
算法层要求设计一种能够适配多任务调用的逆运动学求解方法，并给出合理的训练与推理机制。
系统层要求将视觉、求解器、子任务、调度逻辑和执行接口组织成完整系统。
验证层要求提供任务级实验结果，并对图表配置、指标解释和实验设计合理性进行说明。

2.2 课题问题定义

从系统角度看，本文研究的问题可以定义为：在已知机械臂当前关节状态、目标物体空间位置、预期末端姿态以及任务执行逻辑的前提下，如何构建一套统一系统，使机械臂能够完成感知输入、目标位姿求解、动作分解执行和流程重构，并为后续实机接入预留可替换接口。

将这一问题进一步拆解，可以得到以下几个关键子问题。

如何将多种复杂任务拆解为较稳定且可复用的动作原语。

如何根据目标位置和姿态快速求出对应关节角，并保证求解过程在仿真执行中具备足够稳定性。

如何在不同任务之间共享求解器和原语库，避免为每个任务单独构建独立底层逻辑。

如何将动作序列组织为可扩展、可回退和可重构的流程控制结构。

如何在当前仿真阶段不直接接入实机通信的前提下，先完成命令流逻辑验证，并为后续 IP 传输留出明确替换位置。

2.3 系统设计原则

为了使系统满足本科毕业设计的完整性要求，同时兼顾后续扩展性，本文在总体方案设计中遵循以下原则。

结构化原则。

底层求解器、中间原语库、高层调度与执行桥接层应各自承担明确职责，避免模块边界模糊。

复用性原则。

同一求解器应尽量服务于多类任务，同一原语应尽量被多个任务流程共享。

解耦原则。

任务逻辑不直接依赖具体通信方式，当前输出到文本文件，后续改为 IP 发送时不应破坏任务本体。

可验证原则。

每个层次都需要有明确可观察结果，包括模型误差、任务执行状态、命令输出行数和终点位姿误差。

渐进过渡原则。

在尚未接入真机阶段，先利用文本输出方式验证指令流；待时机成熟后，再将底层发送逻辑替换为网络通信。

2.4 系统总体架构

结合当前代码实现，本文系统总体上采用分层结构设计，可划分为感知层、求解层、原语层、调度层和执行层。
感知层负责获取任务目标在世界坐标系中的空间信息。
当前项目中主要通过视觉模块和场景中的目标体位姿确定目标位置。
对于视觉巡检和装配任务，系统会根据目标点生成扫描起点、装配点和安全偏置点等关键坐标。

求解层由 GNN 逆运动学模型及其封装后的 `GNNIKSolver` 构成。
该层负责将目标末端位姿映射为关节角配置，并通过迭代式修正保证求解结果达到可执行标准。

原语层由多个子任务脚本组成。
当前系统包含 `task_approach.py`、`task_orient.py`、`task_reach.py`、`task_linear.py`、`task_grasp.py`、`task_release.py`、`task_retreat.py`、`task_hover.py`、`task_arc.py` 和 `task_screw.py` 等文件。
这些脚本各自实现一种稳定动作模式，并通过统一参数接口被上层调用。

调度层由三份行为树任务脚本构成，分别是 `task1_bt_demo.py`、`task2_bt_demo.py` 和 `task3_bt_demo.py`。
每个脚本内部都定义了 `Sequence`、`Fallback`、`Condition`、`Action` 和 `Blackboard` 等结构，以组织任务流程和共享黑板变量。

执行层包括 `runtime_bridge.py` 和 `robot_ip_client.py`。
前者负责关节状态应用、仿真前向更新和末端位姿读取，后者负责生成 `movej(...)` 指令字符串并在当前阶段写入文本文件。
该层正是系统从仿真逻辑走向真实通信接口的过渡区。

从数据流角度看，系统工作过程如下。
首先，感知层获得任务目标。
其次，行为树根据任务定义依次调用对应原语。
原语内部根据当前状态和目标生成中间目标位姿，并调用求解器。

求解器返回关节角后，运行时桥接层将其应用到 MuJoCo 模型，同步输出命令字符串并更新末端状态。
之后，黑板变量根据任务结果继续向下推进，直至整个任务完成。

【图2-1 系统总体架构图】
配图说明：建议采用五层分层框图绘制。最上层为感知层，标注“目标位置获取、扫描点生成、装配点确定”；第二层为求解层，标注“GNNIKSolver、GATv2、可微 FK、迭代修正”；第三层为原语层，列出 10 个子任务原语；第四层为调度层，标注三类行为树任务；最下层为执行层，标注“runtime_bridge、RobotIPClient、movej 文本输出、未来 IP 通信”。图中箭头应体现数据流从上至下执行、状态反馈由下至上回传。

2.5 机械臂对象与仿真环境分析

本文选择 UR10e 六自由度机械臂作为研究对象。
选择该平台主要基于以下考虑：UR 系列机械臂结构清晰、资料丰富，适合作为本科阶段的研究对象；UR10e 具有六自由度，既能体现典型工业机械臂的运动学复杂性，又便于结合标准模型在 MuJoCo 中建立仿真环境；该机械臂在抓取、移动、姿态调整和装配等任务中都具有代表性，便于验证多任务系统的通用性。

项目中使用 `universal_robots_ur10e` 目录下的模型文件构建场景。
机械臂场景中定义了主要关节、连杆和末端站点 `attachment_site`，该站点作为计算末端位置与姿态误差的统一参考。
通过 MuJoCo，系统能够在给定关节角后快速进行前向更新，获取所有关节和末端执行器在世界坐标系中的状态。
这为数据生成、求解器验证和任务联调提供了可靠基础。

2.6 多任务流程重构需求分析

在本文中，多任务重构并不意味着随机拼接任意动作，而是指在统一求解器和统一原语库的基础上，通过不同的流程组织方式完成不同任务。

抓取转运任务强调“目标靠近—姿态对准—抓取—搬运—释放—回退”的典型闭环。

视觉巡检任务强调“生成扫描起点—到达扫描位置—圆弧扫描—悬停观察—安全回位”的过程。

装配旋拧任务则强调“装配点接近—姿态轴线对准—接触预压—旋拧进给—退回安全位”的过程。

这三类任务虽然在流程上有所不同，但都共享若干底层动作模式。

例如，`Approach` 可用于抓取前的安全接近，也可用于装配前的粗对准；`Orient` 可用于抓取姿态调整，也可用于巡检相机方向校正和旋拧轴向对齐；`Linear` 可用于直线插入、回 home 和 safe-home 等过程；`Retreat` 可用于抓取后离开目标，也可用于装配后退出工作区。

正因为底层动作具有复用性，所以构建统一原语库才具有意义。

2.7 总体技术路线

围绕上述需求，本文总体技术路线可概括为以下五个阶段。

第一阶段，搭建 UR10e 机械臂仿真环境，明确关节名称、末端站点、目标物体和基础场景配置。

第二阶段，自动生成逆运动学训练数据，建立图数据集和特征标准化方法，完成 GNN 模型训练。

第三阶段，将训练后的模型封装为可在线调用的迭代式求解器，加入安全约束和收敛判定逻辑。

第四阶段，围绕多任务需求构建统一子任务库，并将其组织为行为树任务。

第五阶段，构建运行时桥接层，在当前阶段先输出机械臂脚本格式文本命令，为后续实机 IP 通信迁移预留接口。

【图2-2 本文技术路线图】

配图说明：建议采用纵向流程图方式绘制。从“仿真建模”出发，依次连接“数据生成”“模型训练”“求解器封装”“原语库构建”“行为树组织”“命令输出与验证”。每个阶段下方应附一到两项关键成果，例如“生成配对数据”“保存 best_gat_model.pt”“输出 task1_movej_commands.txt”等，体现工程实现的落地性。

2.8 系统模块与文件组织关系

为了使论文描述与代码实现保持一致，有必要对当前项目目录结构进行梳理。

`GNN` 目录主要负责数据集、模型、训练、求解器与工作空间可视化等内容。

`subtask` 目录主要负责原语实现、行为树任务脚本和运行时桥接层。

仓库根目录下的 `robot_ip_client.py` 负责生成和写出 URScript 风格文本命令，`run_vision.py` 负责视觉相关流程，`real_robot_ip_migration_guide.md` 则说明了当前文本输出向未来 IP 通信迁移的设计思路。

从论文撰写角度看，这种目录结构也体现出系统由算法层向工程层扩展的过程。

2.9 系统接口设计分析

对于一个面向多任务的机械臂系统而言，
真正决定系统是否易于扩展的关键因素，
往往不是某个单独算法的局部精度，
而是不同层次之间的接口设计是否清晰。
如果接口设计混乱，
即使当前脚本可以运行，
后续一旦增加任务数量或改变执行方式，
系统也会迅速陷入维护困难。

本文当前项目之所以能够在同一套基础上支撑抓取转运、
视觉巡检和装配旋拧三类任务，
一个重要原因就在于接口边界相对明确。
从系统结构出发，
至少存在以下四类关键接口。

第一类接口是感知结果到任务变量的接口。
视觉模块并不直接控制机械臂，
而是先产生任务需要的空间目标，
例如 `target_xyz`、
`scan_start_xyz`、
`scan_end_xyz`、
`safe_home_xyz` 等黑板变量。
这种设计使感知模块和动作模块之间不是硬连接关系，
而是通过中间变量间接协同。

第二类接口是任务层到原语层的接口。
行为树的动作节点不会直接操作关节角，
而是将当前黑板中的目标点和当前状态传递给相应原语。
因此，
原语层看到的是统一格式的任务输入，
而不是面向某一特定任务脚本的私有变量。

第三类接口是原语层到求解层的接口。
原语的主要职责是确定“目标位姿应该是什么”，
而不是自己求解关节。
一旦目标位姿确定，
便由求解器统一计算关节解并返回结果。
这使得所有原语都共享同一个求解内核。

第四类接口是求解层到执行层的接口。
求解器并不关心命令最终是写入文本文件、

发送给网络 socket,
还是接入其他控制总线。
它只负责通过桥接层发出“某一阶段需要输出当前关节状态”的请求。
后续执行方式变化时,
只需替换桥接层或客户端实现即可。

上述四类接口的存在,
构成了本文系统能够稳定演进的前提。
从本科毕业设计的角度看,
这类接口分析非常必要。
因为它能够说明项目并不是若干脚本的简单拼接,
而是围绕模块边界进行过思考和整理的系统实现。

进一步从软件工程角度分析,
接口设计的价值还体现在故障定位上。
例如,
若任务失败,
可以先判断是目标变量生成错误,
还是原语内部目标构造错误,
还是求解器收敛失败,
还是执行层命令输出方式存在异常。
这种分层排查能力,
正是模块化系统优于单脚本系统的重要体现。

在后续真实机械臂接入场景中,
接口优势会更加明显。
若当前项目没有提前将执行层抽象出来,
那么从文本命令输出切换到 IP 通信发送时,
所有子任务脚本都可能需要重写。
而现在只要 `RobotIPClient` 的内部发送逻辑完成替换,
求解器、

原语和行为树的主体都可以保持不变。
这正是本文总体方案具有工程可持续性的核心原因之一。

2.10 多任务重构可行性分析

在论文中讨论“多任务重构”时，
不能只停留在概念描述，
还应说明为何本文当前方案确实具备重构基础。

首先，
从动作语义角度看，
本文构建的原语库并不是针对单一任务定制的特殊动作，
而是具有较普遍几何意义的基本操作。
例如，
“靠近”本质上是一种基于目标方向的安全偏置动作；
“对准”本质上是一种姿态校正动作；
“抵达”本质上是一种点位到达动作；
“回退”本质上是一种反方向退出动作。
这些动作语义不依赖具体任务类型，
因此天然适合被不同流程复用。

其次，
从任务结构角度看，
三类任务虽然表面不同，
但都可以分解为“目标生成—接近—对准—执行主体动作—退出或归位”这样的通用结构。
抓取任务中的主体动作是抓取与释放；
巡检任务中的主体动作是扫描与观察；
装配任务中的主体动作是旋拧与退出。
正是由于流程骨架具有相似性，
行为树才能够围绕少量高复用原语构建多个任务。

再次，
从参数配置角度看，
原语之间的复用不仅依赖代码级复用，
还依赖参数级复用。
例如，
同样是 `Approach` 原语，
在抓取任务中可能采用较短的接近距离，
在装配任务中则可能采用更严格的轴线方向约束；
同样是 `Linear` 原语，
在返回 home 和返回 safe-home 时使用的目标点不同，
但其控制方式并未改变。
这说明任务差异主要体现在参数和目标构造上，
而不是体现在底层算法上。

从行为树组织方式看，
多任务重构的可行性还来自于节点层次清晰。
当前脚本中的 `Sequence` 节点强调顺序执行，
`Fallback` 节点强调失败回退，
`Condition` 节点强调状态判断，
`Action` 节点强调原语调用。
这样一来，
任务重构时的主要工作不是重写底层求解器，
而是重新组合这些树节点。
这对于系统扩展非常重要。

在正式论文论证中，
还可以从“低耦合、高复用、参数可配置、路径可重组”四个角度概括多任务重构可行性的来源。
这种写法比单纯说“系统具备良好扩展性”更有说服力。

同时需要指出，
本文当前阶段强调的是“离线任务重构”与“仿真环境中的可执行重

构”,
并不等同于任意场景下的全自主任务生成。
这一界限在论文中应交代清楚。
这样既不会夸大系统能力,
也能更准确地说明当前研究已经完成的工作边界。

2.11 方案可行性与实现风险分析

一个完整的本科毕设方案除了要说明“为什么这样设计”,
也应说明“这样的设计是否真的可行”以及“可能面临哪些风险”。

从技术可行性看,
本文方案的基础较为明确。
首先,
UR10e 的仿真模型已经存在,
MuJoCo 环境能够稳定加载场景并执行前向更新。
其次,
图神经网络训练所需的数据生成流程已经打通,
模型训练与保存也已形成固定流程。
再次,
三类行为树任务已经在当前仓库中完成联调运行,
说明该方案不是停留在纸面上的设想,
而是已经具备实际运行基础。

从工程可行性看,
当前项目采用文本命令输出方式替代真实通信,
使系统能够在不引入设备风险的前提下先完成动作流验证。
这种分阶段推进方式本身就提高了实施成功率。
因为对于本科毕业设计而言,
若在尚未验证上层逻辑时就直接接入真实设备,
反而会让调试难度和安全风险显著增加。

从风险角度分析，
本文方案主要存在三类风险。
其一是数据覆盖风险。
若训练数据覆盖范围不足，
模型在未见工作区中可能出现求解不稳定。
其二是任务参数风险。
若原语参数配置不当，
例如接近距离过小、
回退距离不足或旋拧步数设置不合理，
会导致任务链整体稳定性下降。
其三是接口迁移风险。
当系统从文本输出迁移到 IP 通信时，
尽管整体结构可复用，
但通信时序、
设备响应延迟和异常处理策略仍需重新验证。

这些风险并不会否定方案本身，
但需要在论文中如实说明。
这样既体现出研究态度的严谨性，
也便于后续答辩时说明系统为什么要采取阶段化验证路线。

2.12 本章小结

本章从任务书和开题报告出发，
明确了课题的系统目标和指标要求，
并结合当前代码实现给出了系统总体方案。
在此基础上，
进一步从接口设计、
任务重构可行性、
方案可行性与实现风险等方面对总体方案进行了展开分析。
通过这些讨论可以看出，

本文的重点并不在于单点算法实验，
而在于面向多任务场景建立统一求解器、
统一原语库、
统一行为树组织和统一执行桥接层。
这一总体方案为后续方法设计和工程实现奠定了基础。

【表2-1 系统层次划分与职责说明】

配图说明：本表建议列出感知层、求解层、原语层、调度层和执行层五层结构，分别说明对应文件、输入输出和在系统中的职责。表中应明确 `GNN/ik_solver_api.py` 、 `subtask/runtime_bridge.py` 、 `robot_ip_client.py` 等关键文件，体现论文内容与项目实现的一致性。

第3章 基于图神经网络的逆运动学建模与求解方法设计

3.1 逆运动学问题描述

机械臂逆运动学问题的目标是在已知末端目标位姿的前提下求出满足该目标的关节变量。
对于六自由度机械臂，关节角向量可表示为

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6]^T$$

末端目标位姿由位置向量和旋转矩阵共同描述，记为

$$\mathbf{T}_{target} = \begin{bmatrix} \mathbf{R}_{target} & \mathbf{p}_{target} \\ 0 & 1 \end{bmatrix}$$

其中，$\mathbf{p}_{target} \in \mathbb{R}^3$ 表示目标位置，$\mathbf{R}_{target} \in \mathbb{R}^{3\times3}$ 表示目标姿态。
若机械臂正运动学函数记为 $\mathbf{T}_{end} = f(\mathbf{q})$，则逆运动学问题可以写为 $f(\mathbf{q}) = \mathbf{T}_{target}$。

然而，该问题通常具有映射关系高度非线性、同一目标可能存在多组关节解、在关节限位和任务姿态约束下部分理论可达姿态在工程上未必可执行等特点。
因此，本文并不直接将问题理解为“一次性回归全部关节角”，而是将其设计为“预测关节增量并迭代修正”的求解过程。
这样可以在保留学习能力的同时，通过迭代与前向验证提高求解稳定性。

3.2 UR10e 机械臂运动学建模

为了建立可微正运动学模型，首先需要明确 UR10e 的运动学参数表示方式。
本文采用标准 D-H 参数法描述相邻连杆之间的几何关系。
对第 i 个关节，定义连杆长度 a_i、连杆偏距 d_i、连杆扭角 α_i 和关节变量 θ_i。
从第 $i-1$ 坐标系到第 i 坐标系的齐次变换可写为

$$
{}^{i-1}\mathbf{T}_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

将各关节变换矩阵依次相乘，即可得到末端相对于基座的齐次变换矩阵。
在代码实现中，这一过程被封装在训练脚本中的可微正运动学模块内，并在推理阶段由 MuJoCo 前向更新实现状态一致性校验。

传统 D-H 建模更强调数学推导，而图神经网络强调结构表示。
本文将两者结合，并非要用图模型替代运动学建模本身，而是希望在不丢失机械结构信息的前提下，让神经网络具备对关节拓扑关系的显式感知。
机械臂的链式结构天然适合抽象为图。
其中，每个旋转关节对应一个节点，相邻关节之间的物理连接关系对

应边。
这样，网络在进行消息传递时，相当于沿着真实的机械臂连接关系传播信息，比单纯处理无序向量更符合问题本质。

【图3-1 UR10e 机械臂关节链与图结构映射图】
配图说明：本图建议左侧画出 UR10e 机械臂六自由度示意图，标出 shoulder_pan、shoulder_lift、elbow、wrist_1、wrist_2、wrist_3 六个关节。右侧画出六节点链式图，用箭头或无向边表示节点连接关系。图中应用虚线说明“关节—节点”“连杆连接—边”的对应关系。

3.3 图结构表示与节点特征设计

本文将机械臂表示为图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$，其中 $\mathcal{V}$ 为节点集合，包含 6 个关节节点；$\mathcal{E}$ 为边集合，表示相邻关节之间的连接关系。
在数据集构建中，边索引采用双向连接方式。若存在边 (i, j)，则同时保留 (j, i)，使消息能够在相邻关节之间双向传播。
这样的设计与机械臂实际结构一致，因为前部关节会影响后部连杆位置，而末端目标约束也会反向影响中前段关节的修正方向。

直接使用角度值作为输入会带来周期性问题。
例如，π 与 $-\pi$ 在几何意义上相邻，但在数值上却位于区间两端。
因此，本文将关节角采用正余弦方式编码：$\phi(q_i) = [\sin q_i, \cos q_i]$。
这种编码方式能够将原本一维、存在边界跳变的角度表示映射到连续的二维单位圆上，有利于网络学习角度空间的连续性。

为了使每个节点不仅知道自身角度，还知道自身相对于目标的空间关系，本文在节点特征中引入以下内容：当前关节角的正余弦编码；当前关节所在位置的三维坐标；当前关节姿态矩阵展开后的九维表示；当前关节位置与目标末端位置之间的相对位置误差；当前关节姿态与目标末端姿态之间的相对旋转误差；节点身份 one-hot 编码。
根据项目代码，每个节点最终形成 32 维特征向量，其中前 6 维为节点身份编码，其余维度由标准化后的运动学特征组成。

不同特征量纲差异较大，因此本文在数据集构建中对原始特征执行标准化处理：$\hat{\boldsymbol{x}} = (\boldsymbol{x} - \boldsymbol{\mu})/\boldsymbol{\sigma}$。
项目运行时会将统计量缓存到 `ik_stats.json`，从而在推理阶段快速加载，提高初始化效率。

【图3-2 节点特征组成示意图】
配图说明：建议采用矩形堆叠示意图，从左到右依次画出 one-hot 编码、sin/cos 编码、当前位置、当前姿态、相对位置误差、相对姿态误差等模块。图下方可标注总维度为 32。此图的作用是帮助读者理解输入特征并非单一关节角，而是融合了结构身份和几何信息的复合向量。

3.4 基于 GATv2 的逆运动学网络设计

图神经网络种类较多，包括 GCN、GraphSAGE、GAT 等。
本文选择 GATv2 作为核心消息传递结构，主要基于以下考虑：不同关节对最终末端位姿的影响程度不同；网络需要具备根据上下文动态分配权重的能力；项目已经基于 `torch_geometric.nn.GATv2Conv` 完成实现，并在当前代码中形成稳定结构，论文应与实际代码保持一致。

根据 `model.py`，本文模型输入为每个节点 32 维特征，经多层 GATv2 卷积后得到隐藏表示。
隐藏层维度设置为 256，注意力头数设置为 4，网络层数设置为 4。
每一层之后配合 `LayerNorm` 和 `LeakyReLU` 激活，并保留残差式更新思想，以减轻深层图网络在训练时的退化问题。
在图卷积层之后，模型不直接使用单层线性映射输出，而是增加多层感知机解码头。
这一设计是因为逆运动学从末端空间映射回关节空间本质上属于复杂非线性回归，仅靠线性层往往不足以表征关系。

模型输出不是绝对关节角，而是关节角增量：$\Delta\hat{\mathbf{q}} = [\Delta\hat{q}_1, \Delta\hat{q}_2, \cdots, \Delta\hat{q}_6]^T$。
将其与当前关节状态叠加后，即可得到下一次迭代的关节角估计：

$\mathbf{q}_{k+1} = \mathbf{q}_k + \Delta\hat{\mathbf{q}}_k$。
与直接回归最终解相比，增量式预测更适合与迭代求解器结合，便于引入步长裁剪和收敛判定。

【图3-3 GATv2 逆运动学网络结构图】
配图说明：本图建议分为四部分绘制。第一部分为图输入层，标注“6 个节点，32 维特征”；第二部分为 4 层 GATv2Conv + LayerNorm + LeakyReLU；第三部分为 MLP 解码头；第四部分为 6 维关节增量输出。图中建议注明“heads=4”“hidden=256”“layers=4”等主要超参数。

3.5 可微正运动学约束与自动加权损失设计

若只在关节空间监督网络，模型可能学会逼近训练标签，但未必能保证末端位姿真正精确。
因此，本文在训练中引入可微正运动学约束，将预测得到的关节状态重新映射回位姿空间，并在位置与姿态两个层面对网络施加约束。
项目在 `train.py` 中定义了基于 PyTorch 的正运动学模块。该模块将 D-H 参数写成张量，并以批量方式构建每个关节的变换矩阵。通过连续矩阵乘法，即可得到批量样本对应的末端位置和姿态。
由于这些计算全程处于 PyTorch 计算图中，因此可在反向传播时将末端位姿误差的梯度传回网络参数。这就使“正运动学一致性”真正成为训练过程的一部分。

若预测关节增量为 $\Delta\hat{\mathbf{q}}$，真实标签增量为 $\Delta\mathbf{q}$，当前关节状态为 $\mathbf{q}$，则预测末端位姿与目标末端位姿分别可以写为 $(\hat{\mathbf{p}}, \hat{\mathbf{R}}) = FK(\mathbf{q} + \Delta\hat{\mathbf{q}})$ 和 $(\mathbf{p}, \mathbf{R}) = FK(\mathbf{q} + \Delta\mathbf{q})$。
位置误差可写为 $\mathcal{L}_{pos} = \|\hat{\mathbf{p}} - \mathbf{p}\|_1$，姿态误差可写为 $\mathcal{L}_{rot} = \|\hat{\mathbf{R}} - \mathbf{R}\|_1$。

本文训练过程同时考虑关节误差、位置误差和姿态误差。
若直接人工设定三类损失权重，则会面临不同量纲之间难以平衡的问题。

为此，项目引入了基于不确定性的自动加权多任务损失，在 `train.py` 中通过可学习参数 `log_vars` 实现。其形式为：

$$\mathcal{L}_{total} = \exp(-s_1)\mathcal{L}_{joint} + s_1 + \exp(-s_2)\mathcal{L}_{pos} + s_2 + \exp(-s_3)\mathcal{L}_{rot} + s_3$$

训练过程中，网络不仅更新模型权重，也同步更新这三个参数，从而动态平衡三类损失项的影响。
该方法的意义在于，将原本依赖经验调节的权重选择问题转化为可学习过程，提高训练稳定性。

【图3-4 联合损失函数组成与梯度流向图】
配图说明：建议绘制“网络输出—FK 模块—三类误差—AWL 融合”的示意图。在图中应明确显示关节误差直接来自输出与标签比较，位置误差和姿态误差来自 FK 变换后的末端位姿比较。同时用箭头标注三个 `log_vars` 对应的权重调节作用，突出自动加权的思想。

3.6 迭代式求解器封装设计

训练好的神经网络如果只在离线环境下输入一次特征并输出一次预测，则很难直接用于真实任务。
为了让模型能够被不同原语稳定调用，本文将其封装为 `GNNIKSolver`。
该类的目标是统一加载模型与标准化统计量；在给定当前状态与目标位姿时进行迭代求解；在迭代过程中结合 MuJoCo 状态刷新进行验证；在需要时向执行层发射当前关节命令。

根据代码，求解器初始化时主要完成以下工作：选择 CPU 或 CUDA 设备；加载标准化统计量 `ik_stats.json`；构建固定的双向图边索引；加载训练完成的 `best_gat_model.pt`；加载 UR10e 的 MuJoCo 模型；记录六个关节在 `qpos` 中的地址；记录末端站点和关键连杆 ID。
初始化过程中将统计量缓存到 JSON 文件，能够减少反复加载完整数据集的开销，提高求解器启动速度。

求解器的核心方法为 `solve(...)`。
在每轮迭代中，主要步骤如下：将当前关节角写入 MuJoCo 状态；调用 `mj_forward` 更新正运动学；读取各关节连杆位置和姿态；构建标准化节点特征；调用图神经网络预测关节增量；对关节增量进行步长裁剪；更新关节角并执行关节限位处理；再次前向更新并计算末端误差；若满足阈值则提前结束。
这种“神经网络预测 + 物理前向验证”的结构，使求解器兼具学习能力和执行可控性。

在工程使用中，神经网络若一次给出过大的关节增量，容易引发迭代震荡或姿态跳变。
因此，求解器设置 `STEP_CLIP = 0.2`，即单轮迭代中每个关节的增量被限制在 ±0.2 rad 范围内。
此外，关节更新后还需要执行限位处理。对于具备完整圆形范围的关节，代码使用模运算实现无缝角度折返；对于明确限制在半区或工作区中的关节，则使用裁剪方式保持在安全范围内。
求解器使用位置误差和姿态误差双重阈值作为收敛标准。根据代码设置，位置误差阈值为 0.005 m，姿态误差阈值为 0.005。当两者同时满足时，认为当前结果收敛。

3.7 求解流程的详细分析

为了更清楚地说明本文方法如何从输入目标走向最终关节解，
有必要对求解流程进行细化分析。
在当前系统中，
求解并不是一个孤立的数学公式求值过程，
而是一个包含状态更新、
特征重构、
网络预测、
约束修正和误差验证的闭环过程。

第一步是读取当前状态。
求解器接收到当前关节角后,
并不会立刻直接送入图神经网络,
而是先通过 MuJoCo 更新整条机械臂当前姿态。
这一过程的作用在于获取与当前关节角真正对应的各连杆位姿信息,
从而保证后续特征构造基于真实当前状态而非理想假设。

第二步是生成图节点特征。
求解器在读取到各连杆的位置和姿态后,
逐关节构造节点特征。
在这个过程中,
每个节点不仅拥有本关节的角度编码,
还会引入自身位置、
自身姿态、
目标相对误差等信息。
从求解过程角度看,
这等价于让每个关节在预测前都“知道自己现在在哪里、离目标还差多少、在整条链中处于什么位置”。

第三步是进行网络前向预测。
网络根据节点特征和固定的链式图结构输出 6 维关节增量。
需要强调的是,
这一增量不是对最终答案的完全替代,
而是对“当前状态应该朝哪个方向修正”给出的数据驱动建议。

第四步是执行步长裁剪与关节限位。
这一步是从“学习结果”走向“可执行结果”的关键。
如果没有这一步,
神经网络即便偶然给出过大增量,
也会直接传入下一轮状态更新,
从而引发震荡甚至失控。

因此，
本文求解流程中特别强调安全裁剪和区间限制。

第五步是再次进行前向验证。
更新后的关节角必须重新送回 MuJoCo 场景，
并通过末端站点读取位置和姿态误差。
如果误差已经满足阈值，
则说明本轮求解有效。
若仍不满足，
则继续下一轮迭代。

从这一流程可以看出，
本文求解方法本质上并不是“神经网络替代全部运动学求解”，
而是“神经网络提供修正方向，物理模型负责闭环验证”。
这种组合方式非常适合本科毕业设计。
因为它既体现了数据驱动方法的创新性，
又保留了工程上必要的可控性。

3.8 关键超参数对方法行为的影响

在方法章节中，
除了介绍网络结构本身，
还应说明关键超参数为何这样设置。
因为对于工程化模型而言，
超参数并不是抽象数字，
而是直接影响求解行为的重要因素。

首先是隐藏层维度。
当前模型采用 `hidden_channels=256`。
这一设置意味着图卷积层在中间阶段能够容纳较丰富的特征表达。
若隐藏维度过小，
模型可能难以同时编码局部位姿和目标相对误差信息；
若隐藏维度过大，

则训练成本和显存消耗都会上升。
因此,
当前设置体现的是一种兼顾表达能力与计算代价的折中。

其次是网络层数。
当前采用 4 层 GATv2。
若层数过少,
模型能够传播的信息层次有限,
对较远关节之间的关联建模能力不足;
若层数过多,
则容易带来训练难度增加和信息过平滑问题。
四层结构对于六自由度链式机械臂来说,
已经能够形成较充分的拓扑传播深度。

再次是注意力头数。
当前采用 4 头注意力结构。
多头注意力能够让网络从不同子空间理解邻居节点的重要性。
在机械臂问题中,
不同关节对末端位置与姿态的贡献方式并不相同,
因此多头结构具有合理性。

对于求解器而言,
`MAX_ITER`、
`STEP_CLIP`、
`TOL_POS` 和 `TOL_ROT` 等参数同样重要。
`MAX_ITER` 决定单次求解允许的最大修正轮数。
`STEP_CLIP` 决定每轮修正的幅度。
`TOL_POS` 和 `TOL_ROT` 则决定系统何时认定结果已足够接近目标。
这些参数共同决定了求解器的收敛速度、
稳定性和最终精度。

从论文表述角度看，
对这些超参数进行解释还有一个作用，
即帮助读者理解“为什么本文不是随意堆砌一个网络结构，而是围绕具体任务需求进行配置”。
这会明显提高方法章节的说服力。

3.9 方法对多任务系统的适配性分析

一个常见误区是，
只要某个逆运动学模型在离线数据上误差较小，
就认为它自然适合多任务系统。
事实上，
这两者并不完全等价。
真正适合多任务系统的方法，
还必须满足若干额外条件。

第一，
方法应具备统一输入输出格式。
本文求解器只需要当前关节角、
目标位置和目标姿态，
因此所有原语都可以围绕这一接口构造调用。
若求解器需要额外依赖大量任务特定参数，
其复用性就会显著下降。

第二，
方法应具备可重复调用能力。
多任务系统中的动作通常是串联的，
一个原语执行完成后，
下一个原语会以当前状态为起点继续调用求解器。
因此，
求解器必须能够在多次短调用条件下保持稳定。
本文的增量式迭代结构正适合这一点。

第三,
方法应具备与原语层的职责分离。
本文求解器不负责决定该去哪个任务点,
原语也不负责关节级求解。
这种职责分离保证了任务层与方法层之间的边界清晰,
使得方法真正能够成为“被多任务系统共享的底层能力”。

第四,
方法应具备一定的执行接口友好性。
在当前代码中,
求解器保留了与命令发射相关的钩子接口,
能够在求解阶段与桥接层协同。
这一设计虽然看起来只是工程实现细节,
但实际上对系统级联调非常重要。

因此,
从“统一接口”“可重复调用”“职责分离”“执行友好性”四个角度来看,
本文方法并不是一个脱离系统的单独模型,
而是一个专门面向多任务系统组织方式设计的底层求解内核。

3.10 方法局限与边界说明

为了使论文论证更加客观,
本章还需要说明当前方法的适用边界。

首先,
本文方法是在 UR10e 六自由度机械臂及其当前工作空间约束条件下建立的。
这意味着其有效性与当前模型结构、
参数范围和数据分布密切相关。
若机械臂构型发生显著变化,
虽然整体思路仍可迁移,
但具体模型和数据集仍需重新构建。

其次,
本文使用的姿态表示和误差度量方式适合当前任务,
但在更复杂的高精度装配场景中,
可能还需要更细粒度的姿态误差描述或接触约束设计。

再次,
本文求解器强调在线可调用和工程稳定,
因此在一些极端全局随机目标场景下,
未必追求理论上最强的泛化指标。
这也是为什么本文实验章节将重点放在任务链闭环验证,
而不是把全部篇幅投入到全空间随机测试上。

最后,
本文方法目前仍属于“仿真闭环与命令流验证阶段”,
尚未把真实通信链路完全接入。
因此,
论文在表述时必须将“当前已完成工作”和“后续可扩展方向”清楚区分。

3.11 本章小结

从研究逻辑上看,
本文第3章方法设计具有以下合理性:
图结构建模与机械臂链式结构相匹配;
正余弦编码解决了角度周期跳变问题;
可微 FK 使网络训练不再脱离物理约束;
自动加权损失提升了多目标联合优化的稳定性;
增量式迭代求解与任务系统的在线调用需求相一致;
求解器接口设计自然支持与原语层和执行层对接。

本章围绕逆运动学求解方法展开,
给出了从机械臂运动学建模、
图结构表示、

节点特征设计、
GATv2 网络构建、
可微正运动学约束、
自动加权多任务损失、
求解流程分析、
超参数影响、
系统适配性到方法边界说明的完整过程。
这为后续系统工程实现奠定了底层方法基础。

【表3-1 节点特征组成表】
配图说明：建议列出 one-hot 编码、sin/cos 编码、当前位置、当前姿态、相对位置误差、相对姿态误差等字段，并说明各自维度。

【表3-2 GNN 求解器主要参数表】
配图说明：建议列出 `hidden_channels=256`、`num_layers=4`、`heads=4`、`STEP_CLIP=0.2`、`MAX_ITER=50`、`TOL_POS=0.005`、`TOL_ROT=0.005` 等参数，说明其在训练与推理中的作用。

第4章 系统工程实现与控制策略部署

4.1 系统工程实现目标

若将第3章视为方法设计层，则第4章的重点就是说明方法如何真正变成可运行的系统。
本科毕设论文在这一部分不能只简单罗列文件名，而应回答两个问题：方法如何在代码中被落实；系统为何采用当前这种实现方式。
因此，本章围绕数据生成、模型训练、求解器封装、视觉模块、子任务原语、行为树任务组织和执行桥接层依次展开。

4.2 数据集生成流程设计

数据集是训练图神经网络求解器的基础。
若数据分布不合理，模型再复杂也难以得到稳定结果。
逆运动学数据集生成的关键在于，既要覆盖有效工作空间，又要尽量减少多解混淆和不可执行姿态。
项目中数据生成功能由 `generate_ur10e_data.py` 实现。
根据训练目标，每条样本需要包含当前关节角、各连杆当前位置、各连杆当前姿态、目标末端位置、目标末端姿态以及从当前姿态到目标姿态的关节增量标签等信息。

在实践中，若完全随机地在理论全空间中采样，不仅容易产生大量难以收敛的样本，还可能引入多解和异常姿态。
因此，项目在数据生成时对关键关节范围进行了人工限制。
例如，肩部、肘部和腕部部分关节被限定在较稳定的工作区间内。
这样做的目的不是缩小问题本身，而是先在当前课题阶段建立“可控、稳定、可验证”的任务工作空间。
对于本科毕设而言，这种设计是合理的，因为课题重点在于多任务系统组织和统一求解框架，而不是无限扩大全空间泛化能力。

为了兼顾大范围跳跃与小步精调，数据生成采用多尺度采样思想。
具体而言，部分样本对应较小增量，用于提高网络在局部精细调节阶段的性能；部分样本对应中等尺度变化，模拟一般任务中的位姿转换；另有部分样本对应较大幅度变化，用于提高模型应对较大初末状态差异时的能力。
在 `run_pipeline.py` 中，默认样本规模被设置为 40000，生成后的数据保存为 `ur10e_paired_data.npy`，供后续训练使用。

【图4-1 数据生成流程图】
配图说明：建议从“随机关节采样”开始，依次连接“范围约束”“MuJoCo 前向更新”“提取关节与末端状态”“构造样本”“保存 NPY 文件”。图中应明确体现“多尺度采样”和“工作空间约束”两个关键步骤。

4.3 数据集组织与模型训练实现

`dataset.py` 负责将原始 NPY 数据转换为图神经网络可直接读取的 `Data` 对象。
在实现上，该模块主要完成读取原始数组数据、根据机械臂结构构建固定 `edge_index`、对每个样本逐关节构建节点特征、计算训练集均值和标准差、返回标准化后的特征张量以及标签等任务。
这种组织方式保证了训练脚本与数据格式之间的清晰边界，训练脚本无需关心原始数组如何拼接，只需直接调用数据集即可。

为了提高求解器初始化效率，标准化统计量在训练后会被缓存到 `ik_stats.json`。
这样，求解器在推理阶段只需读取该文件，即可完成特征标准化所需参数的准备，而无需再次遍历全部训练数据。
这一设计虽然简单，但体现出明显的工程意识。

模型训练主要由 `train.py` 完成。
脚本包括参数解析、数据加载、网络构建、可微 FK 模块构建、损失计算、优化器更新、验证集评估和最佳模型保存等内容。
训练过程中，使用 AdamW 优化器，以验证集位置误差作为最佳模型保存依据。
这一做法体现出任务导向思路，因为末端位置误差比单纯训练损失更能直接反映求解器在机械臂执行中的有效性。

根据 `run_pipeline.py`，当前训练流程的关键超参数包括 `epochs = 200`、`hidden = 256`、`layers = 4`、`heads = 4`、`dropout = 0.05`。
其中，较低的 dropout 设置是结合数值回归任务特点所做出的工程选择。
对于机械臂逆运动学这种强调连续几何映射的问题，过高的 dropout 反而可能破坏特征传递稳定性，因此当前代码采用较低 dropout，是与任务性质相匹配的。

【图4-2 模型训练流程图】

配图说明：本图建议从“读取 NPY 数据”开始，经过“构建图数据”“前向传播”“计算 joint/pos/rot 三项损失”“AWL 融合”“反向传播”“保存最佳模型”。图中应标出 `best_gat_model.pt` 和 `ik_stats.json` 两个最终产物。

4.4 求解器工程封装

`ik_solver_api.py` 是连接模型训练结果与任务执行系统的关键文件。

在本文系统中，其作用不仅是“调用模型得到结果”，更是“为不同任务提供统一求解接口”。

在面向多任务的系统中，如果每个原语都自行管理模型加载和迭代逻辑，不仅会造成代码重复，还会让系统维护变得困难。

因此，将求解逻辑统一封装在 `GNNIKSolver` 中具有明显必要性。

求解器输入包括当前关节角、目标末端位置、目标末端姿态、最大迭代次数、收敛阈值和步长裁剪参数。

求解器输出包括最终关节角数组和是否收敛的布尔状态。

这种输入输出形式简洁明确，便于不同原语统一调用。

求解器内部在每次迭代前后都会调用 MuJoCo 的 `mj_forward` 更新状态。

这样做的好处在于，能够以仿真环境中的真实模型为准获取各连杆位置与姿态，也能够在求解过程中以末端站点实际状态计算误差，而不是仅依赖模型内部的理想输出。

当前求解器在迭代过程中还支持发射命令。这意味着求解器不仅为原语返回最终角度，还可以在 `solver_iteration` 或 `solver_final` 阶段将关节状态传递给底层桥接层。

4.5 视觉模块与目标生成逻辑

本文系统中的视觉模块主要用于生成任务目标的世界坐标信息。
在抓取转运任务中，视觉目标对应被抓取对象位置；在巡检任务中，视觉目标不仅给出目标中心，还用于计算扫描起点和扫描终点；在装配任务中，视觉目标用于确定装配位置和旋拧轴线方向。
因此，视觉模块并不是单独存在的附加组件，而是行为树黑板中的关键输入来源。

在三个行为树任务脚本中，均定义了 `Blackboard` 类。
其作用是保存任务过程中共享的数据，例如 `target_xyz`、`place_xyz`、`scan_start_xyz`、`scan_end_xyz`、`home_xyz`、`safe_home_xyz` 以及任务完成标志位。
通过黑板变量，不同动作节点无需彼此直接依赖，只需访问共享状态即可。这增强了任务流程的可扩展性。

【图4-3 视觉目标到黑板变量映射图】
配图说明：可用箭头表示视觉模块输出目标点后，如何分别写入 `target_xyz`、`scan_start_xyz`、`scan_end_xyz`、`safe_home_xyz` 等变量。图中应体现黑板作为任务共享状态中心的作用。

4.6 子任务原语库设计

原语库是连接底层求解器与高层任务调度的关键中间层。
若没有原语抽象，则行为树只能直接操作关节或位姿目标，任务逻辑会变得过于底层。
通过原语封装，可将常用动作抽象为有明确语义的操作单元。
当前系统一共实现了 10 类原语，覆盖了典型操作需求，这已经超过任务书中“至少 5 类子任务”的要求。

`Approach` 的作用是在目标点前方一定距离生成安全接近点。
`Orient` 用于在当前位置或目标附近调整末端方向。
`Reach` 强调到达指定空间点。
`Linear` 强调沿插值轨迹进行连续运动。
`Grasp` 与 `Release` 用于模拟末端执行器的夹持和释放操作。

`Retreat` 用于从目标附近沿既定方向退离。

`Hover` 用于在某一点悬停等待。

`Arc` 主要用于视觉巡检任务中的圆弧扫描。

`Screw` 是最具复杂性的原语之一，用于装配任务中的旋拧进给。

当前各原语的 `execute(...)` 接口形式基本一致，一般包括 solver、model_mj、data_mj、viewer、current_angles、current_pos、current_rot、target_xyz 等参数。
这种统一接口设计是原语能够被行为树灵活调用的重要基础。

【表4-1 子任务原语库功能说明表】
配图说明：建议列出 10 个原语名称、核心功能、典型使用场景、主要输入参数和输出结果。该表可直接用来回应任务书中“至少 5 类子任务”的指标要求。

4.7 行为树任务脚本实现

`task1_bt_demo.py` 负责实现抓取转运任务。
该任务逻辑大致为：靠近目标、姿态对准、到达抓取点、执行抓取、抓取后回退、移动到放置点、释放目标、放置后回退。
这一流程具有典型的抓取作业结构，也是最能体现子任务复用性的任务之一。

`task2_bt_demo.py` 负责视觉巡检任务。
该任务会根据视觉目标生成扫描起点和扫描终点，随后组织 `Approach`、`Orient`、`Reach`、`Hover`、`Arc`、`Linear` 等原语完成扫描流程。
巡检任务与抓取任务相比，更强调姿态保持与轨迹连续性，这说明系统不仅支持“抓取—放置”类型操作，也支持以观察和扫描为核心的任务模式。

`task3_bt_demo.py` 负责装配旋拧任务。
在该任务中，系统会依次完成粗接近、姿态对准、到达装配点、预抓

取或预压、旋拧执行、释放、回退以及移动到 safe-home。
从行为树角度看，该任务最能体现原语复用与流程重构的价值，因为它既复用了抓取任务中的接近、对准和回退逻辑，又增加了专门的 `Screw` 原语。

【图4-4 任务1 抓取转运行为树结构图】
配图说明：建议用树状结构画出 `Sequence` 主干，其中包含 `approach`、`orient`、`reach`、`grasp`、`retreat after grasp`、`move to place point`、`release`、`retreat after place` 等节点。图中应突出条件节点对“是否已抓取”和“是否已释放”的判断。

【图4-5 任务2 视觉巡检行为树结构图】
配图说明：建议画出从目标定位到扫描流程的行为树，其中重点节点包括 `approach scan start`、`orient for scan`、`reach scan start`、`hover for focus`、`arc scan`、`hover for data return`、`return home`。图中可用不同颜色区分姿态调整类动作与扫描类动作。

【图4-6 任务3 装配旋拧行为树结构图】
配图说明：建议突出 `screw-in assembly` 节点的核心位置，并展示前后原语之间的关系。图中还应画出 `retreat after assembly` 和 `move to safe home`，体现装配完成后的安全退出逻辑。

4.8 运行时桥接层设计与文本输出接口

在系统发展过程中，若每个原语都各自负责写入 MuJoCo 状态、同步 viewer、读取末端位姿并输出控制命令，则会造成重复代码和执行逻辑分散。
更重要的是，若未来将输出方式从“文本文件记录命令”切换为“IP 通信发送命令”，则所有原语都需要同时修改。
因此，当前项目在 `subtask/runtime_bridge.py` 中建立了运行时桥接层，将这些通用过程统一封装。

根据代码，该模块主要提供 `joint_qpos_addresses(...)`、`apply_joint_positions(...)`、`emit_motion_step(...)`、`apply_motion_step(...)` 和 `get_end_effector_pose(...)` 等函数。其中，`apply_motion_step(...)` 是最关键的函数，它完成关节角写入、MuJoCo 前向更新、命令发射、viewer 同步和可选延时等过程。这样，原语层就无需重复实现这些底层操作，只需聚焦于目标生成和求解调用。

当前 `robot_ip_client.py` 虽然类名中包含 IP Client，但实现上并未直接建立网络通信连接，而是采用一种更适合当前阶段的方式：将 URScript 风格的 `movej(...)` 指令逐行写入文本文件。
该文件的作用在于让开发者能够清晰观察任务过程中机械臂将执行哪些关节动作。
当前代码支持三种 `send_mode`：`solver_iteration`、`linear_step` 和 `solver_final`。
这意味着系统可以灵活选择在哪个阶段把关节状态转化为外部命令。
从工程角度看，当前文本输出模式的意义主要在于先验证任务脚本是否能生成正确命令流，先验证各原语与求解器之间的调用顺序是否合理，并在未接入实机时避免因为网络传输或设备执行问题干扰上层逻辑验证。

【图4-7 文本命令输出到 IP 通信迁移示意图】
配图说明：左侧画出当前流程：行为树任务 -> 原语 -> 求解器 -> runtime_bridge -> RobotIPClient -> 文本文件。右侧画出未来流程：行为树任务 -> 原语 -> 求解器 -> runtime_bridge -> RobotIPClient -> Socket/IP -> 真实机械臂。两条流程前半段保持一致，突出“仅替换最后一段发送逻辑”的设计优势。

4.9 原语调用流程标准化分析

在工程实现章节中，
如果只说明“系统有若干原语文件”，

仍然不足以解释原语库为何能够稳定支撑多任务。
因此，
还需要进一步分析原语调用流程为何能够保持统一。

从当前代码结构看，
各原语在形式上都围绕同一类输入组织：
求解器对象、
MuJoCo 模型与数据对象、
当前关节状态、
当前末端状态以及目标点。
这种统一接口设计带来的直接收益有三点。

第一，
任务层不必为每个原语编写完全不同的调用语句。
行为树的 `Action` 节点只要知道“调用哪个原语”和“从黑板中读取哪个目标点”，
就可以完成任务组织。

第二，
原语之间便于互换和替换。
如果后续需要新增某个动作，
只要其遵守现有接口风格，
就能较容易地融入当前系统。
这说明原语库不仅是“代码集合”，
更是“接口规范集合”。

第三，
有利于调试。
当任务失败时，
可以围绕统一输入格式检查问题来源，
而不必面对多个互不相同的调用方式。

从论文表达角度看，
这一部分非常适合作为“系统工程化程度”的证据。
因为本科论文若只写“我实现了很多原语”，
说服力仍然有限；
而若进一步说明这些原语为何可以形成统一库，
则工程价值会更加明确。

4.10 黑板机制与行为树协同分析

行为树只是任务组织框架，
黑板则是行为树能够在多步流程中共享状态的重要基础。
当前三个任务脚本均定义了 `Blackboard` 类，
并在其中维护目标点、
回位点、
任务标志位等共享信息。

黑板机制的作用主要体现在以下几个方面。

第一，
实现信息集中管理。
若没有黑板，
各动作节点之间只能通过参数层层传递状态，
会导致接口迅速复杂化。
黑板的存在使得关键状态只需要集中维护一次。

第二，
提高流程可读性。
在阅读行为树脚本时，
研究者可以较清楚地知道系统有哪些核心共享变量，
例如抓取任务中的 `target_xyz` 和 `place_xyz`，
巡检任务中的 `scan_start_xyz` 和 `scan_end_xyz`，
装配任务中的 `safe_home_xyz` 等。

第三，
为条件判断节点提供基础。
行为树中的 `Condition` 节点需要根据当前任务状态作出判断，
例如是否已抓取、
是否已扫描、
是否已完成装配。
这些状态都需要黑板来保存。

第四，
为后续扩展留出空间。
若未来加入更多传感信息、
异常标志或执行反馈，
最自然的做法也是继续扩展黑板对象，
而不必重写全部任务逻辑。

因此，
黑板并不是行为树脚本中的一个普通数据类，
而是任务组织层与执行反馈层之间的重要连接点。
在论文中对其进行说明，
可以明显增强系统设计的层次感。

4.11 文本命令输出格式的工程意义

当前项目中一个很容易被忽视、
但实际上非常重要的设计，
就是将 `movej(...)` 字符串逐行写入文本文件。
如果只从最终结果看，
这种做法似乎只是一个临时替代方案。
但从工程推进路径看，
它具有十分明确的阶段价值。

首先，
文本命令输出让动作流变得可见。

机械臂任务脚本内部虽然会不断调用求解器和原语,
但如果没有对外可观察结果,
就很难确认任务到底做了什么。
文本文件将每一步关节指令显式记录下来,
使系统行为具备了可检查性。

其次,
文本输出天然适合与实验章节结合。
例如,
本文任务1、
任务2、
任务3 分别输出 82、
72、
164 行命令。
这一统计结果不仅能反映任务复杂度,
也能反映不同原语和不同发送模式对动作流规模的影响。

再次,
文本输出是连接仿真与实机的重要中间层。
如果一开始就直接发送网络命令,
系统调试会同时受到算法、
任务逻辑、
通信时序和设备执行等多重因素影响。
而当前做法等于先把“上层动作是否正确”这个问题单独验证清楚,
之后再处理“如何把动作发送到真机”这一问题。

最后,
文本输出还有利于论文展示。
相较于单纯说“后续可接入真实机械臂”,
展示标准化的 `movej(...)` 输出格式更能说明当前系统已经完成了命令层抽象。

4.12 系统工程完整性评价

综合本章内容可以看到,
本文项目已经形成了一条较完整的工程链路。

在数据层,
系统能够自动生成受控工作空间样本,
并完成图数据构建与标准化。

在模型层,
系统能够完成图神经网络训练、
损失优化和最佳模型保存。

在求解层,
系统能够将训练结果封装为可在线调用的迭代式求解器。

在动作层,
系统形成了较完整的原语库,
覆盖接近、
姿态校正、
抵达、
直线运动、
抓取、
释放、
回退、
悬停、
圆弧扫描和旋拧等典型动作。

在任务层,
系统已经用行为树构建了三类代表性任务。

在执行层,
系统已经通过桥接层和文本命令客户端打通了命令输出流程。

换言之，
本文项目已经不再是单纯的“算法脚本”，
而是具备较清晰软件层次和扩展接口的系统雏形。
对于本科毕业设计而言，
这正是论文价值的重要来源。

4.13 本章小结

本章围绕系统工程实现展开，
详细说明了数据生成、
数据集组织、
训练流程、
求解器封装、
视觉模块、
子任务原语、
行为树任务、
黑板机制、
执行桥接层以及文本命令输出接口的实现思路。
通过这一章可以看出，
本文项目已经形成了较完整的软件架构。
后续实机部署时，
主要需要替换的是最底层发送通道，
而无需重写任务逻辑和求解逻辑。

【表4-2 关键工程文件与功能对应表】
配图说明：建议按目录列出 `generate_ur10e_data.py`、`dataset.py`、`model.py`、`train.py`、`ik_solver_api.py`、`runtime_bridge.py`、`robot_ip_client.py`、`task1_bt_demo.py`、`task2_bt_demo.py`、`task3_bt_demo.py` 等文件的职责。

第5章 实验设计与结果分析

5.1 实验章节设计原则

本科毕设中的实验章节不能只追求“数值越多越好”，更重要的是实验设计要与课题目标一致。
本课题的核心是“面向多任务的机械臂智能运动规划”。
因此，实验设计必须围绕以下三个层次展开：底层求解器是否具备可调用性和基本精度；子任务原语是否具备复用性和稳定性；复合任务能否在统一系统中执行成功。
若只给出网络训练损失曲线，则无法体现多任务系统价值；若只展示任务动画截图，又不足以说明底层方法的有效性。
因此，本章采用“系统级任务验证为主、方法支撑分析为辅”的结构更为合理。

5.2 实验平台与运行环境

当前实验主要在 MuJoCo 3 环境下完成，代码运行对象为 UR10e 机械臂仿真场景。
训练部分依赖 `PyTorch` 与 `PyTorch Geometric`，任务联调部分依赖 `mujoco`。
需要说明的是，本文实验重点在于验证系统设计和任务闭环能力。因此，仿真平台的作用不是替代真实机械臂，而是作为当前阶段的验证环境。在后续接入实机时，本文通过运行时桥接层和 `RobotIPClient` 已为通信替换提供基础。

5.3 实验评价指标设计

根据课题目标，本文实验采用如下评价指标：任务是否成功完成；末端最终位置误差；末端最终姿态误差或旋转矩阵误差；单次任务输出的命令条数；任务总耗时；多次重复运行下的稳定性。
这些指标之所以合理，原因在于它们分别对应系统的不同层面。任务成功与否反映行为树和原语协同是否正确；末端误差反映求解器和运动执行结果是否满足精度要求；命令条数反映当前文本输出接口是否

按预期工作，并可间接反映任务路径复杂程度；耗时反映系统执行效率；多次重复运行稳定性则反映当前参数配置是否稳健。

末端位置误差定义为 $e_p = \|\mathbf{p}_{final} - \mathbf{p}_{target}\|_2$。
在当前任务验证中，主要使用旋转矩阵均方误差表示姿态对准程度：
$e_r = \frac{1}{9}\|\mathbf{R}_{final} - \mathbf{R}_{target}\|_F^2$。
稳定性可用重复运行成功率表示：$\eta = \frac{N_{success}}{N_{total}} \times 100\%$。

5.4 任务1：抓取转运实验结果

抓取转运任务用于验证系统在“目标接近—姿态调整—抓取—转运—释放”这一典型工业流程中的闭环能力。
该任务与现实中的上下料、分拣和零件转运场景高度相关，也是多任务系统中最基础但最具代表性的流程之一。
任务主要步骤为：读取目标位置；调用 `Approach` 靠近目标；调用 `Orient` 调整末端方向；调用 `Reach` 到达抓取点；调用 `Grasp` 完成夹持；调用 `Retreat` 从抓取点回退；调用 `Reach` 到达放置点；调用 `Release` 释放目标；调用 `Retreat` 从放置点回退。

根据当前联调结果，任务1 连续运行 3 次均成功完成，成功率为 100%。
每次任务结束后，放置点末端误差均为 0.0 mm。
每次任务生成的文本命令条数为 82 行。
单次任务运行时间约为 0.65 s 至 0.68 s。
这一结果说明，在当前工作空间和参数配置下，抓取转运任务能够稳定完成。尤其是放置点误差达到 0.0 mm，说明关键定位环节满足较高精度要求，可作为回应任务书中“子任务控制精度”目标的重要依据之一。

【图5-1 抓取转运任务执行过程截图】
配图说明：建议由 4 至 6 张子图组成，分别展示靠近、对准、抓取、搬运、放置、回退等关键阶段。每张子图下方可标注对应原语名称，体现原语与任务流程的对应关系。

【表5-1 抓取转运任务重复运行结果表】

配图说明：表中建议列出 3 次运行编号、是否成功、放置点误差、命令条数、耗时。表尾可给出平均值，突出任务稳定性。

5.5 任务2：视觉巡检实验结果

视觉巡检任务用于验证系统在“目标感知—扫描路径生成—姿态保持—圆弧扫描—回 home”流程中的执行能力。

与抓取转运相比，该任务更强调连续扫描和姿态稳定，因此能够更好地考察 `Arc` 与 `Hover` 原语的作用。

任务2 的主要流程如下：获取视觉目标；计算扫描起点与终点；`Approach` 靠近扫描起点；`Orient` 调整观察方向；`Reach` 到达扫描起始位姿；`Hover` 稳定观察；`Arc` 执行圆弧扫描；`Hover` 完成数据停留；`Linear` 返回 home。

这一任务体现出系统能够围绕目标自动生成派生路径点，而不是只执行单点到达动作。

当前联调结果显示，视觉巡检任务连续运行 3 次均成功完成，成功率为 100%。

每次任务结束回到 home 位置时，末端误差为 2.5638 mm。

每次任务输出 72 行 `movej` 命令字符串。

单次运行时间约为 0.56 s 至 0.58 s。

巡检任务的终点误差略高于抓取转运任务，这是因为任务中包含圆弧扫描和悬停观察过程，动作链较长，同时任务结束时的误差是多个连续动作累积后的结果。但从毫米级精度和重复稳定性来看，该任务仍然达到了较好的系统联调效果。

更重要的是，该任务证明系统不仅能处理“接近—抓取”类动作，还能处理扫描和姿态跟随任务，这对于说明多任务能力非常关键。

【图5-2 视觉巡检任务扫描轨迹图】

配图说明：建议绘制末端在扫描过程中的三维轨迹，并用起点和终点

标记说明 `scan_start_xyz` 与 `scan_end_xyz`。若可获取俯视图和侧视图，可在一页中排布两种视角，更直观体现圆弧扫描效果。

【表5-2 视觉巡检任务重复运行结果表】

配图说明：建议列出 3 次运行的成功状态、home 误差、命令条数、耗时等指标，并在表后说明巡检任务更关注姿态稳定和扫描连续性。

5.6 任务3：装配旋拧实验结果

装配旋拧任务是本文三类复合任务中最复杂的一项。

其不仅需要完成目标接近和姿态对准，还需要在接触条件下执行轴向进给和旋转复合动作。因此，该任务最能反映系统在复杂原语条件下的稳定性和重构能力。

任务3 的主要流程如下：获取装配目标；`Approach` 接近装配点；`Orient` 调整轴向方向；`Reach` 到达装配点；`Grasp` 完成预抓或预压；`Screw` 执行旋拧进给；`Release` 释放；`Retreat` 回退；`Linear` 移动至 safe-home。

这一流程说明，装配任务并不是完全独立的新任务，而是在已有原语基础上增加 `Screw` 这一专用动作。这正体现了“在统一原语库基础上实现任务重构”的思想。

当前联调结果表明，任务3 连续运行 3 次均成功完成，成功率为 100%。

每次任务结束回到 safe-home 时，末端误差为 3.0453 mm。

每次任务输出 164 行命令字符串。

单次运行时间约为 13.57 s 至 14.27 s。

在 `Screw` 原语阶段，日志显示失败步数与总步数之比为 36/90，最终位置误差为 1.72 mm，旋转矩阵均方误差为 0.00002。

从结果来看，虽然旋拧过程较抓取和巡检任务更复杂，但系统仍然能够完成整体任务闭环。

装配任务耗时明显长于前两个任务，其原因在于 `Screw` 原语内部需要

分步执行旋拧进给，并且在每一步中都进行状态更新和校验。这与任务复杂度是相匹配的。

【图5-3 装配旋拧任务关键阶段截图】
配图说明：建议展示靠近装配点、轴线对准、旋拧中间过程、旋拧完成、回退与 safe-home 五个阶段。图中可使用箭头标出旋拧方向和进给方向，增强说明性。

【表5-3 装配旋拧任务重复运行结果表】
配图说明：建议列出 3 次运行的成功状态、safe-home 误差、命令条数、耗时，并在表下注明 `Screw` 原语阶段的最终位置误差与旋转误差。

5.7 三类任务综合对比分析

为了更清晰地说明多任务系统的整体表现，有必要对三类任务进行横向比较。
从任务性质看，抓取转运强调点位精确和流程完整；视觉巡检强调姿态稳定与连续扫描；装配旋拧强调复杂原语和连续进给控制。
从运行结果看，三类任务均达到 100% 的三次重复成功率，说明当前系统在给定工作空间和参数条件下具备较好的稳定性。
从误差看，抓取转运的末端精度最高，巡检与装配任务由于动作链更长，误差略大，但仍保持在毫米级范围内。
从命令条数看，装配旋拧任务输出最多，表明其底层执行步骤最复杂。
从耗时看，装配旋拧也显著长于前两类任务，这与其复杂工艺特征一致。

综合来看，这些结果可以支撑以下结论：统一求解器能够被三类不同任务共享；统一原语库具有较高复用度；行为树调度能够将不同原语重新组织成不同任务流程；文本命令输出接口可以稳定记录各任务的动作流。

【表5-4 三类任务综合结果对比表】

配图说明：建议将任务名称、重复运行成功率、最终误差、命令条数、任务耗时五项指标放在同一张表中，便于横向比较。

5.8 实验实施过程说明

为了提高实验部分的可复现性，
仅给出最终统计结果是不够的，
还应说明实验是如何实际开展的。

当前三类任务实验均遵循相似的实施流程。
首先，
确认 MuJoCo 场景模型加载正常，
确保 `scene.xml` 或相关场景入口文件可被任务脚本正确读取。
其次，
加载训练完成的 `best_gat_model.pt` 以及标准化统计量文件，
确认求解器能够正常初始化。
再次，
根据任务类型设置控制面板参数，
例如接近距离、
方向向量、
回退距离、
插值步数、
旋拧圈数等。
随后，
启动行为树任务脚本，
观察任务执行状态、
输出日志和命令文本文件生成情况。
最后，
记录任务是否成功、

最终误差、
命令条数和耗时等指标。

在本科论文中加入这类实施流程说明有两个好处。
一是使实验章节更具操作性，
避免看起来像一组孤立数字。
二是便于答辩时说明“这些结果不是凭空写出的，而是按固定步骤验证得到的”。

对于三类任务来说，
其差异主要体现在参数配置和原语组合上，
而不是体现在底层实验流程上。
这种统一实施方式也从侧面证明了系统结构的统一性。

5.9 图表组织与结果呈现建议

实验章节的说服力，
不仅取决于结果本身，
还取决于图表组织是否合理。
对于本文而言，
图表安排应服务于“系统闭环能力”这一核心论点。

第一类图表应体现任务流程。
例如三类行为树结构图、
任务关键阶段截图等，
它们主要用于说明系统究竟做了什么。

第二类图表应体现运动过程。
例如巡检轨迹图、
旋拧误差变化图、
抓取与放置关键帧等，
它们主要用于说明任务执行是否平稳、是否符合预期。

第三类图表应体现统计结果。
例如各任务重复运行结果表、
三类任务综合对比表、
命令条数统计表等，
它们主要用于说明系统是否稳定可复现。

在正式排版时，
建议每一类任务至少包含一张过程图和一张结果表。
这样的结构最符合工科本科论文的阅读习惯。
若图过多而表过少，
论文会显得偏演示化；
若表过多而图过少，
则不利于展示系统的工程直观性。

此外，
图注和表注应尽量写得具体。
例如，
不宜只写“任务结果图”，
而应写成“抓取转运任务中靠近、抓取、搬运与放置过程示意”。
这样更具学术写作规范性。

5.10 结果讨论

从当前实测结果出发，
本文系统至少体现出三个方面的特点。

第一，
统一求解器具备较好的任务共享能力。
三类任务均依赖同一 `GNNIKSolver`，
但最终都能够稳定完成，
说明求解器没有被限制在某一单独任务中使用。

第二，
原语库具备较高复用度。
在三类任务中，
`Approach`、
`Orient`、
`Reach`、
`Linear`、
`Retreat` 等原语被重复使用，
且未因任务不同而需要重写底层逻辑。
这正是“子任务重构能力”的直接体现。

第三，
执行桥接层的设计是有效的。
虽然当前尚未接入真实机械臂，
但三类任务都能够稳定生成对应文本命令流，
说明执行层抽象已经工作。

如果进一步比较三类任务的差异，
还可以发现不同任务对系统提出的侧重点并不相同。
抓取转运更强调关键点位精度，
因此放置误差 0.0 mm 的结果具有较强代表性。
视觉巡检更强调轨迹连续性和姿态稳定性，
因此圆弧扫描和回 home 的整体平稳性更值得关注。
装配旋拧更强调复杂原语下的长期稳定执行，
因此其较长耗时和较多命令条数本身就是合理现象，
并不能简单理解为性能下降。

因此，
在论文中讨论结果时，
不能把三类任务仅仅当作“数值对比对象”，
而应强调它们分别代表不同能力侧面。
这种讨论方式会比简单罗列结果更符合论文写作规范。

5.11 结果可信度与边界说明

在实验部分，
除了报告正面结果外，
还应说明结果的可信度来源和适用边界。

当前结果可信度主要来自以下几个方面。

第一，
所有结果都来源于当前仓库中的实际任务脚本运行，
而不是凭空假设或脱离代码的理论推演。

第二，
三类任务均进行了重复运行，
而不是只截取一次最优结果。
虽然重复次数还不算很多，
但对于当前阶段的本科毕设验证来说，
已经能够说明任务流程不是偶然成功。

第三，
结果中既有任务成功状态，
也有位置误差、
姿态误差、
命令条数和耗时等多个维度，
因此不是单一指标支撑结论。

但与此同时，
结果也存在明确边界。
当前实验在已知仿真环境、
已知目标设置和受控工作空间中进行，
因此其主要证明的是“当前系统在目标工作域内能够稳定执行”，
而不是“模型已经在任意未知场景下达到最优泛化水平”。

这种边界说明对于本科论文非常重要。
因为它能够体现作者对研究完成度和适用范围的清醒判断，
也避免在答辩中被质疑为夸大结果。

5.12 实验设计的合理性讨论与局限性分析

在本科毕设写作中，实验设计不仅要呈现结果，还要说明“为什么这样设计”。
对于本文而言，实验设计采用任务级验证为主，具有如下合理性：课题主题是多任务机械臂运动规划，因此最终评价标准应是“任务是否完成、流程是否稳定、原语是否可复用”，而不是仅仅比较神经网络在某一测试集上的单纯回归误差；任务书和开题报告都强调系统集成和任务重构，因此论文实验章节必须体现系统级闭环；当前代码已经稳定支持三类行为树任务运行，这些实验结果是真实可复现的，比编造大量不必要的理论对比数据更可靠；当前系统尚处于仿真向实机过渡的阶段，文本命令输出正是实验中应当重点说明的工程验证内容之一。

尽管当前系统已经完成了三类任务联调，但仍存在一些值得进一步改进的地方。
第一，当前实验主要在受控工作空间内进行，模型泛化能力仍与训练采样空间存在关联。若扩展到更大范围和更复杂姿态集合，仍需进一步完善数据覆盖与模型鲁棒性。
第二，当前任务验证以系统级结果为主，对比实验和消融实验尚不充分。若后续时间允许，可以补充不同网络结构、不同损失设置或不同步长裁剪参数下的对比。
第三，当前执行接口尚停留在文本命令输出阶段，虽然这正符合现阶段设计目标，但距离真实机械臂在线控制仍有一步之遥。
第四，装配旋拧任务已经能够完成基本流程，但在接触过程建模、力控与误差补偿方面仍有进一步提升空间。
指出这些局限并不是否定当前工作，而是为了使论文表达更加客观。
对于本科毕业设计而言，既要展示成果，也要说明边界。

5.13 图表配置建议与本章小结

为了使论文正式排版后具有较强说服力，本章建议配置以下图表：三类任务行为树结构图；三类任务关键执行截图；巡检任务扫描轨迹图；旋拧任务进给过程示意图；任务结果综合对比表；命令输出文件示意图。

其中，命令输出文件示意图尤为重要，因为它能够直接体现当前“文本输出替代真实通信”的工程设计思想。

【图5-4 三类任务命令输出文件示意图】

配图说明：建议截取 `robot_command_output` 目录中 `task1_movej_commands.txt` 、 `task2_movej_commands.txt` 、 `task3_movej_commands.txt` 的部分内容。图中应显示每条命令为一行，突出当前接口已经输出标准化 URScript 风格指令，便于后续替换为 IP 发送。

【图5-5 巡检任务末端三维轨迹图】

配图说明：如果后续从仿真中导出末端位置序列，可绘制三维轨迹曲线，并标注起点、终点和弧线扫描段。该图有助于体现 `Arc` 原语的连续轨迹性质。

【图5-6 旋拧任务末端位姿误差变化图】

配图说明：建议横坐标为旋拧步数，纵坐标分别为位置误差和旋转误差。可以采用双纵轴或上下两幅子图方式。图中应显示随着旋拧进行，位置误差保持在可接受范围内，旋转误差维持较小水平。

本章围绕系统级任务验证展开，

从实验设计原则、

实验实施过程、

评价指标、

三类任务结果、

综合对比、

结果讨论、

可信度说明、
图表建议和局限性分析等方面进行了说明。
当前结果表明，
本文系统已经能够在统一求解器与统一原语库支持下完成抓取转运、
视觉巡检和装配旋拧三类典型任务，
并通过文本命令输出方式形成可观察、
可验证的执行结果。

【表5-5 论文正式排版建议图表清单】

配图说明：建议按章节列出应放置的图表名称、作用、素材来源和建议页内位置，方便后续排版时快速整理。

第6章 总结与展望

6.1 全文工作总结

本文围绕面向多任务的机械臂智能运动规划问题，以 UR10e 六自由度机械臂为对象，完成了从方法设计到系统实现、从算法训练到任务联调的完整研究过程。

在研究思路上，本文首先从任务书和开题报告中提炼出“多任务重构”“统一求解”“系统集成”和“实机过渡接口”四个核心要求，并以此构建总体技术路线。

在方法设计上，本文将机械臂关节链表示为图结构，构建了融合局部位姿与目标相对误差的节点特征，基于 GATv2 建立逆运动学预测模型，并引入可微正运动学约束和自动加权多任务损失机制，从而增强模型的结构表达能力和物理一致性。

在工程实现上，本文完成了训练数据自动生成、数据集标准化、模型训练、求解器封装、视觉目标处理、统一原语库构建、行为树任务组织以及运行时桥接层设计。

在系统验证上，本文实现了抓取转运、视觉巡检和装配旋拧三类典型任务，并在 MuJoCo 环境下完成多次重复运行验证。实验结果表明，

系统能够稳定输出动作流并完成任务闭环，说明本文提出的系统方案具有较好的可行性。

6.2 本文主要结论

结合前文分析与实验结果，本文主要得到以下结论：机械臂关节链采用图结构表示是合理可行的，有助于表达关节之间的拓扑依赖关系；通过正余弦编码和相对位姿特征设计，可以提升网络对目标空间关系的表达能力；可微正运动学约束和自动加权多任务损失机制能够增强训练过程的稳定性和物理一致性；将神经网络封装为迭代式求解器，并结合步长裁剪、关节限位和前向验证，有利于提高在线调用稳定性；统一子任务原语库和行为树调度方式能够有效支撑多任务流程重构；运行时桥接层和文本命令输出机制，为后续从仿真过渡到实机 IP 通信提供了明确接口基础。

6.3 后续工作展望

虽然本文已经完成了当前阶段的系统构建与验证，但仍有若干方向值得进一步深入。

第一，可进一步扩大数据采样范围，提高模型对更广工作空间和更复杂姿态的泛化能力。

第二，可增加更多任务类型，例如双目标协同抓取、多工位切换、障碍绕行等，进一步检验原语库和行为树框架的扩展性。

第三，可在 `RobotIPClient` 中将当前文件写入逻辑替换为 socket 发送逻辑，完成与真实机械臂控制器的通信联调。

第四，可引入力控、接触检测和误差补偿策略，提升装配类任务的精细化水平。

第五，可补充更系统的对比实验与消融实验，使论文在方法论层面的论证更加完整。

6.4 本章小结

本章对全文工作进行了总结，并从模型泛化、任务扩展、通信迁移和实机部署等方面给出了展望。总体来看，本文已经在本科毕业设计范围内完成了一套较完整的多任务机械臂智能运动规划系统，并为后续进一步研究打下了基础。

【图6-1 本文工作总结与后续拓展方向图】
配图说明：建议左侧概括本文已完成工作，包括 GNN 求解器、原语库、行为树、文本命令输出。右侧列出未来方向，包括 IP 通信、真实机械臂部署、更多任务类型和更强泛化能力。

参考文献

[1] Denavit J, Hartenberg R S. A kinematic notation for lower-pair mechanisms based on matrices[J]. Journal of Applied Mechanics, 1955, 22(2): 215-221.

[2] Craig J J. Introduction to Robotics: Mechanics and Control[M]. 4th ed. Boston: Pearson, 2018.

[3] Siciliano B, Khatib O. Springer Handbook of Robotics[M]. 2nd ed. Cham: Springer, 2016.

[4] Lynch K M, Park F C. Modern Robotics: Mechanics, Planning, and Control[M]. Cambridge: Cambridge University Press, 2017.

[5] Todorov E, Erez T, Tassa Y. MuJoCo: A physics engine for model-based control[C]//2012 IEEE/RSJ International Conference on Intelligent Robots and Systems. 2012: 5026-5033.

[6] Kingma D P, Ba J. Adam: A method for stochastic optimization[C]//International Conference on Learning Representations. 2015.

[7] Loshchilov I, Hutter F. Decoupled weight decay regularization[C]//International Conference on Learning Representations. 2019.

[8] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[C]//International Conference on Learning Representations. 2018.

[9] Brody S, Alon U, Yahav E. How attentive are graph attention networks? [C]//International Conference on Learning Representations. 2022.

[10] Kendall A, Gal Y, Cipolla R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics[C]//2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2018: 7482-7491.

[11] Colledanchise M, Ögren P. Behavior Trees in Robotics and AI: An Introduction[M]. Boca Raton: CRC Press, 2018.

[12] Fey M, Lenssen J E. Fast graph representation learning with PyTorch Geometric[J]. ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[13] Universal Robots. UR10e Technical Specifications[EB/OL].

[14] Universal Robots. Script Manual: e-Series and UR-Series[EB/OL].

[15] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.

[16] LaValle S M. Planning Algorithms[M]. Cambridge: Cambridge University Press, 2006.

[17] Khalil W, Dombre E. Modeling, Identification and Control of Robots[M]. Oxford: Butterworth-Heinemann, 2002.

[18] Spong M W, Hutchinson S, Vidyasagar M. Robot Modeling and Control[M]. 2nd ed. Hoboken: Wiley, 2020.

[19] 王田苗, 陶永. 现代机器人技术导论[M]. 北京: 机械工业出版社, 2021.

[20] 熊有伦, 丁汉. 机器人技术基础[M]. 武汉: 华中科技大学出版社, 2019.

[21] 项目任务书、开题报告及相关项目代码文档资料[Z].

附录A 图表与实验占位说明

本附录仅保留正式排版时仍可能需要补充的图表与实验占位，不再展开大段模块说明，以免附录篇幅过大影响正文主体比例。

【图A-1 UR10e 仿真场景总图】
说明：用于展示机械臂、目标物和末端站点的空间关系。

【图A-2 节点特征组成图】
说明：用于辅助解释第3章中的图结构输入设计。

【图A-3 三类任务行为树总览图】
说明：用于在正式终稿或答辩 PPT 中集中展示任务组织方式。

【图A-4 文本命令输出示意图】
说明：用于展示当前 `movej(...)` 文本输出格式，为后续 IP 通信过渡提供直观说明。

【实验A-1 更多重复次数下的任务稳定性统计】
说明：如后续时间允许，可将三类任务的重复次数由 3 次扩展为 10 次以上，以进一步增强统计说服力。

【实验A-2 不同发送模式下的命令输出规模对比】
说明：如后续需要更细化说明通信层策略，可比较 `linear_step`、`solver_final` 和 `solver_iteration` 三种模式。